Machine Learning I

Tema 2. Aprendizaje Supervisado

2.4 Otros enfoques

Profesor: Raquel Martinez & Jose Tomás Palma

Curso 2024-2025

Índice de contenidos

1 Clasificador k-NN

2 Aprendizaje por Cuantización de Vectores

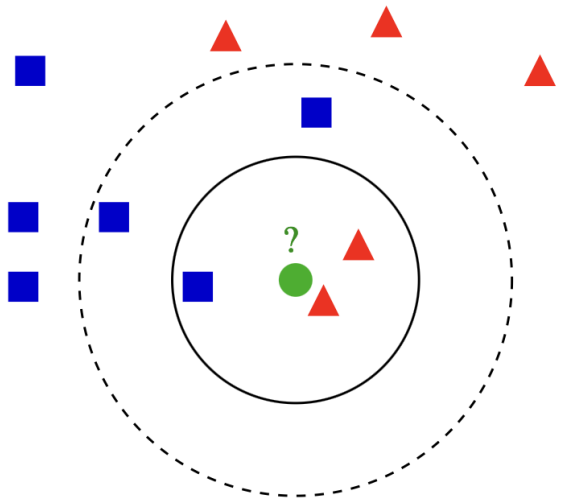
Clasificador k-NN

Clasificador k-NN

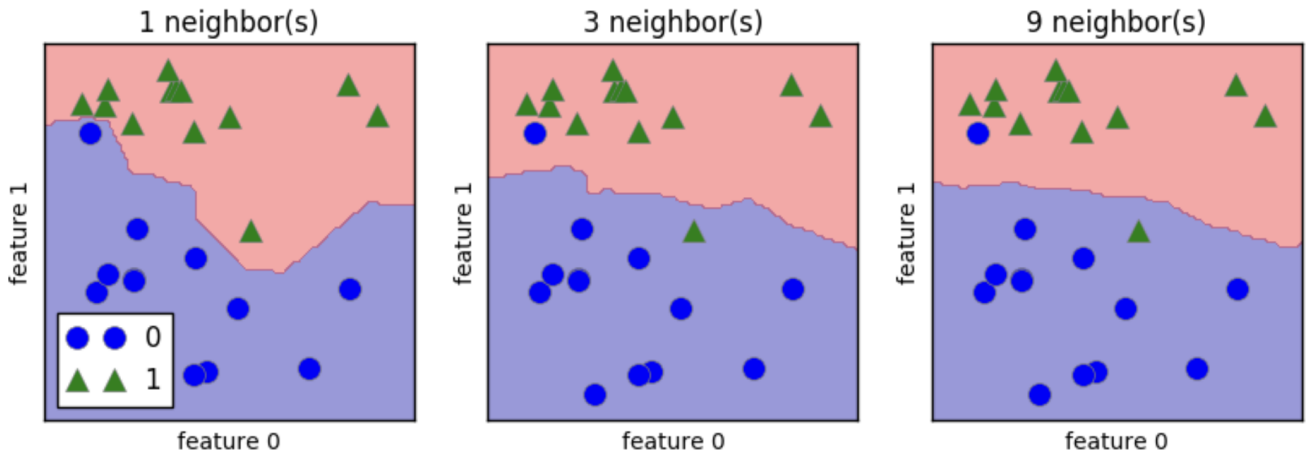
El algoritmo de los k vecinos más cercanos (k-NN, o k Nearest Neighbour) es un algoritmo de clasificación supervisado basado en criterios de vecindad.

Características:

- No hay entrenamiento
- El clasificador está constituido por el conjunto de entrenamiento y el valor de k
- La entrada **x** se asigna a la clase mayoritaria de las k muestras más cercanas del conjunto de entrenamiento.



Existe un **compromiso** en la elección de k



Clasificador k-NN

- Se puede utilizar cualquier medida de distancia.
- En caso de empate entre dos o más clases necesitamos una regla heurística que rompa los empates:
  - Seleccionar la clase que contenga al vecino más próximo.
  - Seleccionar la clase con distancia media menor.
  - Seleccionar la clase más probable a priori.
  - .....
- Para seleccionar el valor de K, hay que tener en cuenta que empíricamente se ha demostrado que el porcentaje de casos bien clasificados es no monótono respecto a de K.
- Una buena elección es elegir valores de K entre 3 y 7.

Clasificador K-NN: variaciones

- **K-NN con rechazo.** Se puede dejar un objeto sin clasificar si no se cumplen unas garantías mínimas.
  - El número de votos debe superar un umbral mínimo
  - Establecer algún tipo de mayoría absoluta
- **K-NN con distancia media.** Asignar un nuevo objeto a aquella clase cuya distancia media sea menor.
- **K-NN con distancia mínima.** Asignar un nuevo objeto a la clase cuyo representante esté más cercano.
  - Se requiere elegir un representante por clase, que generalmente es el objeto más cercano a su centro de gravedad.

Claisficador K-NN: variaciones

- **K-NN con pesado de casos seleccionados.** Se pueden ponderar los casos seleccionados de tal forma que no tengan la misma importancia a la hora de realizar la asignación de clases.
  - Se puede hacer que el peso sea inversamente proporcional a la distancia al objeto a clasificar.
- **K-NN con pesado de variables.** Ahora lo que se intenta es que a la hora de calcular la distancia, las características no contabilicen de igual forma, sino que tengan un peso asociado de acuerdo con su relevancia para la clasificación. En este caso la distancia se calcularía de la siguiente forma:

$$d(x, x_j) = \sum_{k=1}^{n} w_k (x_k, x_{rk})^2$$

Clasificador K-NN

Conclusiones

- **Ventajas**:
  - El coste de aprendizaje es nulo.
  - No necesitamos hacer ninguna suposición sobre los conceptos a aprender.
  - Es muy tolerante al ruido.
  - Puede extenderse para problemas de regresión.
  - Fácil de implementar
- **Inconvenientes**:
  - El coste de encontrar los K vecinos es grande (kd-trees).
  - No hay un mecanismo genérico para decidir el mejor valor de K (hiperparámetro).
  - Su rendimiento baja si el número de características es grande.
  - Su interpretabilidad es nula.

Índice de contenidos

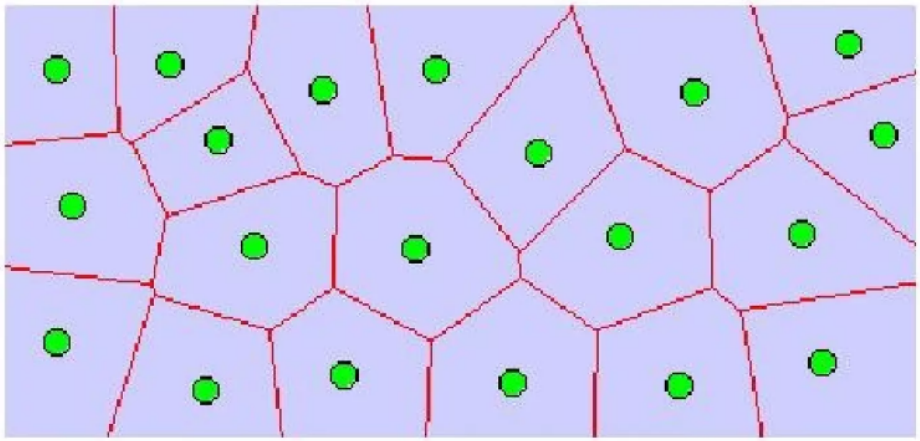
1 Clasificador k-NN

2 Aprendizaje por Cuantización de Vectores

LVQ ("Learning Vector Quantization")

Qué son

- Son una forma de redes neuronales artificiales que se utilizan principalmente en problemas de clasificación supervisada.
- En lugar de intentar aprender una función continua que mapee los datos de entrada a las etiquetas de salida, un LVQ divide el espacio de entrada en regiones discretas y asigna etiquetas de clase a cada región.
- Añaden entrenamiento supervisado tras la autoorganización (VQ), lo que aporta mayor potencia manteniendo la rapidez.



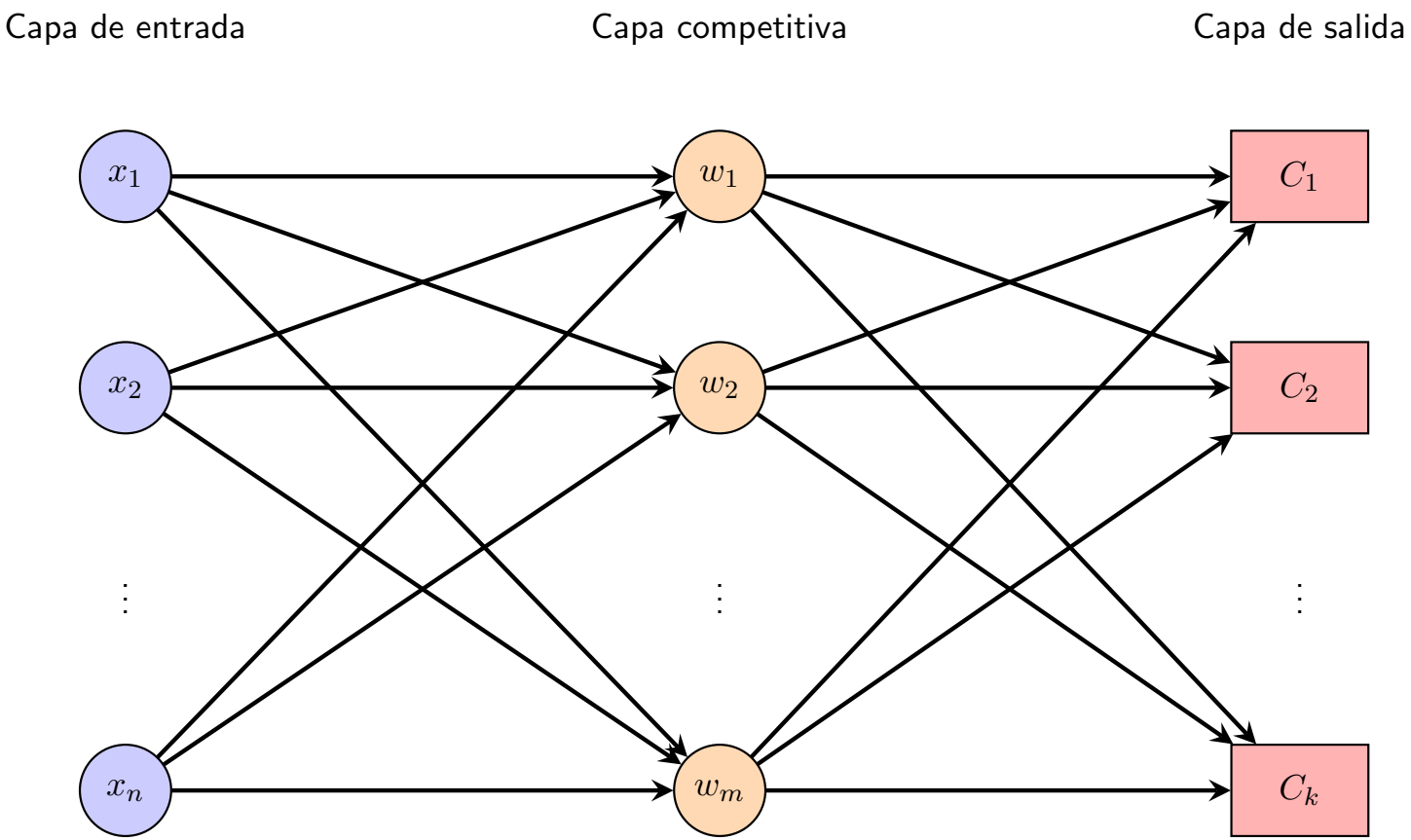
LVQ ("Learning Vector Quantization")

Descripción y diagrama

- Una LVQ se considers un tipo de red neuronal un poco particular.
- Sua arquitectura de LVQ consta de tres capas principales:
  - **Capa de entrada**: Recibe los patrones de entrada (vectores de características).
  - **Capa competitiva**: Contiene los vectores prototipo que representan las clases $\omega_{\mathbf{i}}$.
  - **Capa de salida**: Asigna la clase correspondiente al prototipo ganador.
- Como veremos aprenden por mediante el ajuste de pesos.

LVQ (Learning Vector Quantization)

Descripción y diagrama



LVQ ("Learning Vector Quantization")

Entrenamiento

- **Entrenamiento:**
  - El proceso de entrenamiento de un LVQ implica presentar repetidamente ejemplos de entrenamiento al modelo y ajustar los vectores de referencia $\mathbf{w_i}$ (también llamados prototipos) asociados con cada categoría.
  - Los prototipos representan puntos en el espacio de entrada que representan las características de cada categoría.
  - Durante el entrenamiento, los prototipos se mueven gradualmente hacia los ejemplos de entrenamiento correspondientes, de modo que se ajusten a los datos de entrada de manera óptima.
- Una vez que se ha completado el entrenamiento, el LVQ se puede utilizar para clasificar nuevos ejemplos asignándolos a la categoría representada por el prototipo más cercano en el espacio de entrada. Por ejemplo utilizando la distancia euclídea como media.

LVQ ("Learning Vector Quantization")

Entrenamiento

LVQ1: Aprendizaje competitivo

Sólo se actualiza el vector de la neurona ganadora g, es decir, aquella que tiene más "similitud" con el ejemplo de entrada. Si la clase de $\mathbf{x}$ coincide se refuerza el valor, sino se aleja. α es la tasa de aprendizaje.

$$\mathbf{w}_g(k+1) = \begin{cases} \mathbf{w}_g(k) + \alpha(k)[\mathbf{x}(k) - \mathbf{w}_g(k)] & \text{si } \mathbf{x}(k) \text{ está bien clasificado} \\ \mathbf{w}_g(k) - \alpha(k)[\mathbf{x}(k) - \mathbf{w}_g(k)] & \text{en caso contrario} \end{cases}$$

- Los prototipos se alejan de los bordes interclases y se obtiene una aproximación a la clasificación Bayesiana (sin parametrizar).
- La tasa de aprendizaje $\alpha(k)$ es un valor entre 0 y 1 que decrece con el tiempo.

LVQ ("Learning Vector Quantization")

LVQ2

- El cambio fundamental respecto al algoritmo original radica en que LVQ2 se centra en ajustar dos prototipos cuando el vector de entrada $\mathbf{x_i}$ está cerca de la frontera de dos clases.

Condición de actualización de prototipos

LVQ2 sólo actualiza los prototipos $\mathbf{w_i}$ y $\mathbf{w_j}$ si se cumplen las siguientes condiciones:

- El patrón de entrenamiento $\mathbf{x_k}$ está cerca de la frontera de dos clases.
- El prototipo más cercano $\mathbf{w_i}$ pertenece a la clase incorrecta.
- El prototipo más cercano $\mathbf{w_j}$ pertenece a la clase correcta.

- Regla de actualización:

$$\mathbf{w}_i(k) = \mathbf{w}_i(k) - \alpha(k)[\mathbf{x}(k) - \mathbf{w}_i(k)]$$
$$\mathbf{w}_j(k) = \mathbf{w}_j(k) + \alpha(k)[\mathbf{x}(k) - \mathbf{w}_j(k)]$$

LVQ ("Learning Vector Quantization")

LVQ2.1

- **Problema del LVQ2:**
  - Los prototipos tienden a aproximarse cada vez más ya que el prototipos ganador se aleja menos de la entrada que lo que se acerca el otro prototipos.
- **Solución: LVQ2.1**
  - Es un refinamiento del LVQ2 para evitar que los prototipos se aproximen cada vez más.
  - Sólo se actualiza si la entrada está en una **ventana pre-establecida** entre los dos prototipos ganadores cuya anchura se define como

$$\text{mín}\left(\frac{d(\mathbf{x},\mathbf{w}_j)}{d(\mathbf{x},\mathbf{w}_i)},\frac{d(\mathbf{x},\mathbf{w}_i)}{d(\mathbf{x},\mathbf{w}_j)}\right) > \frac{1-\beta}{1+\beta}, \qquad \beta \sim 0.25$$

LVQ ("Learning Vector Quantization"

LVQ2.1

- **Problema del LVQ2.1:** Tiende a alejar demasiado los prototipos de la frontera:
  - Cuando el prototipos más cercano es de la misma clase que la entrada, se produce un alejamiento de los prototipos tras la actualización
  - Cuando el prototipos más cercano es de clase distinta, se produce un acercamiento de los prototipos tras la actualización
  - Como el primer caso se da más frecuentemente que el segundo, ...
- Por esto, se debe usar con un valor pequeño del parámetro de aprendizaje y un número bajo de iteraciones (30 a 200 veces el número de muestras).

LVQ ("Learning Vector Quantization")

LVQ3: Actualización prototipos

- **Caso 1:** Un prototipo pertenece a la clase correcta y otro a la incorrecta.
  - **Se procede igual que para LVQ2.**
- **Caso 2:** Ambos prototipos pertenecen a la clase correcta.
  - **Ambos prototipos se mueven hacia el patrón de entrenamiento.**

$$\mathbf{w}_i(k+1) = \mathbf{w}_g(k) + \varepsilon\alpha(k)[\mathbf{x}(k) - \mathbf{w}_i(k)]$$
$$\mathbf{w}_j(k+1) = \mathbf{w}_g(k) + \varepsilon\alpha(k)[\mathbf{x}(k) - \mathbf{w}_j(k)]$$

- **Caso 3:** Ambos prototipos pertenecen a la clase incorrecta.
  - **Ambos prototipos se mueven alejándose del patrón de entrenamiento.**

$$\mathbf{w}_i(k+1) = \mathbf{w}_g(k) - \varepsilon\alpha(k)[\mathbf{x}(k) - \mathbf{w}_i(k)]$$
$$\mathbf{w}_j(k+1) = \mathbf{w}_g(k) - \varepsilon\alpha(k)[\mathbf{x}(k) - \mathbf{w}_j(k)]$$

- Generalmente, ε suele ser pequeño $\varepsilon \in [0.1, 0.5]$

LVQ ("Learning Vector Quantization")

Inicialización de los prototipos

- **Inicialización de los prototipos**
  - **Número inicial:** proporcional al número de muestras de cada clase.
  - **Valores iniciales:**
    - Valores aleatorios dentro del rango de los datos de entrenamiento.
    - Seleccionan muestras reales del conjunto de entrenamiento (estratificada o no).
    - Usando Clustering.
    - La media de los datos de cada clase

LVQ ("Learning Vector Quantization")

Recomendaciones

- Se recomienda comenzar con el LVQ1 optimizado (OLVQ1): muy rápido e introduce una tasa de aprendizaje individual por prototipo adaptativa
  - **Aumenta** si el prototipo ganador es de la clase de la entrada.
  - **Disminuye** si el prototipo ganador es de la clase de la incorrecta.
  - Normalmente converge rápido (30-50 épocas).
- Después, si se desea mejorar la clasificación y el tiempo no es crítico, se puede emplear el LVQ1, LVQ2.1 ó el LVQ3
  - Se emplea un valor bajo del "learning rate".
- **¿Cuándo parar?**:
  - Hay que evitar el sobreentrenamiento
  - Para ello, se establece el número de épocas entre 50 y 200.
  - El número exacto sólo se encuentra por la experiencia y depende de los datos de entrada. (Se puede emplear también validación cruzada).